



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Machine Learning Assignment Report

SIMATS Engineering

Assignment – ITA0606 Machine Learning

**Assignment Title - Design and Development of an
Intelligent Real-Time Traffic Congestion Prediction
System Using Machine Learning**

Team Members

Sk. Arshad Basha (192312336)

Deepan Kumar P (192312498)

Sanjaykumar C (192424143)

Acknowledgement and Abstract

Acknowledgement

I express my sincere gratitude to the faculty members and department for providing the opportunity to complete this project. I also acknowledge the open-source machine-learning ecosystem, particularly Python, NumPy, pandas, Matplotlib and scikit-learn, which provides the tools required to build and evaluate predictive systems.

Abstract

Urban traffic congestion causes travel delays, increased fuel consumption, additional emissions, driver frustration, and reduced road-network efficiency. This project proposes an intelligent real-time traffic congestion prediction system that uses machine-learning techniques to classify traffic conditions as Low, Medium, or High congestion. The proposed system accepts traffic-related features such as vehicle count, average speed, road occupancy, weather condition, incident status, hour of day, and weekday/weekend status. A preprocessing stage cleans and scales numerical data where required, followed by model training and validation.

Two supervised-learning approaches are considered: Logistic Regression as an interpretable baseline and Random Forest as the final predictive model. In the experimental demonstration, a reproducible synthetic traffic dataset containing 1,200 observations is used to validate the workflow. The Random Forest model provides the stronger classification performance in the experiment and also supports feature-importance analysis. The system can be extended to live traffic APIs, CCTV-derived vehicle counts, IoT sensors, GPS traces, and a dashboard for traffic operators.

Table of Contents
1. Problem Statement and Problem Formulation
2. Objectives and Expected Outcomes
3. Requirements, Constraints and Assumptions
4. Application of Relevant Course Knowledge / Concepts
5. Design / Proposed Solution / Methodology
6. Algorithm, Pseudocode and Flowchart
7. Implementation / Source Code and Environment / Tools
8. Test Cases and Expected / Actual Results
9. Execution Screenshots / Outputs
10. Results and Validation
11. Analysis, Comparison, Trade-offs and Justification
12. Broader Considerations / SDG Relevance
13. Conclusion, Limitations and Possible Improvements
14. Individual Contribution of Group Members
15. References
16. One-Page Individual Reflection

1.1 .Problem Statement and Problem Formulation

Problem Statement

Traffic congestion is a dynamic problem in which road conditions can change rapidly because of traffic volume, road capacity, weather, accidents, special events, lane closures, and time-of-day effects. Traditional traffic monitoring systems often describe current conditions but may provide limited predictive intelligence. A predictive system can estimate the upcoming congestion state and support proactive traffic management.

The problem addressed in this project is the design and development of a machine-learning system that receives current or near-real-time traffic observations and predicts a categorical congestion level. The system should be sufficiently fast for operational use, interpretable enough to support decision-making, and flexible enough to accept data from sensors, APIs, or historical traffic records.

Problem Formulation

Let each traffic observation be represented by a feature vector $X = [h, d, w, v, s, o, i]$, where h is hour of day, d represents weekday/weekend, w is weather category, v is vehicle count, s is average speed, o is road occupancy, and i indicates an incident. The target y belongs to $\{\text{Low, Medium, High}\}$. The learning objective is to estimate a function $f(X) \rightarrow y$ that minimizes classification error while maintaining low inference latency.

A practical objective can be written as: minimize $L(y, f(X))$ subject to prediction time and resource constraints. For a production system, the loss function can also be weighted so that false negatives for High congestion receive greater cost.

2.Objectives and Expected Outcomes

Objectives

- Develop a machine-learning pipeline for real-time traffic congestion classification.
- Identify and preprocess traffic features that have predictive value.
- Train and compare an interpretable baseline model and a tree-based ensemble model.
- Evaluate the models using accuracy, precision, recall, F1-score and a confusion matrix.
- Provide a design that can be connected to real-time traffic sensors or APIs.
- Create visual outputs that help traffic operators understand predicted congestion.
- Document constraints, assumptions, risks, ethical considerations and future extensions.

Expected Outcomes

Outcome	Expected Evidence
Functional prediction pipeline	Input $\rightarrow$ preprocessing $\rightarrow$ prediction $\rightarrow$ output class
Reliable classification	Accuracy, precision, recall and F1-score
Model comparison	Baseline vs Random Forest table and graph
Operational usability	Low-latency prediction architecture
Interpretability	Feature-importance discussion
Scalability	Architecture allowing API/IoT integration
Academic reproducibility	Complete Python source code and test cases

3.Requirements, Constraints and Assumptions

Functional Requirements

- The system shall accept traffic observations containing the required features.
- The system shall validate numerical ranges and categorical values.
- The system shall preprocess data consistently during training and inference.
- The system shall train a classification model using labeled historical data.
- The system shall output Low, Medium, or High congestion.
- The system shall report model evaluation metrics.
- The system should support repeated predictions without retraining.

Non-Functional Requirements

- Prediction should be fast enough for near-real-time operation.
- Training code should be reproducible using a fixed random seed.
- The system should be maintainable and modular.
- Input data and predictions should be logged for auditing.
- The dashboard should present information clearly without unnecessary complexity.

Constraints

- The demonstration uses synthetic data rather than a live city traffic feed.
- Traffic patterns vary by city, road geometry, weather, and special events.
- Model quality depends strongly on data quality and label quality.
- Real-time deployment requires a data-ingestion layer and monitoring infrastructure.
- Sensor outages, API delays, privacy rules and network failures may affect predictions.

Assumptions

- Historical observations are representative enough to learn useful patterns.
- Input variables are measured consistently.
- Congestion labels can be derived from a defined traffic rule or historical ground truth.
- Training and deployment feature definitions remain consistent.

4.Application of Relevant Course Knowledge / Concepts

The project applies multiple concepts from Artificial Intelligence, Machine Learning, Data Analytics, Software Engineering and Intelligent Systems.

Concept	Application in Project
Supervised Learning	Learning a mapping from traffic features to congestion classes.
Classification	Predicting Low, Medium or High congestion.
Feature Engineering	Constructing useful variables such as hour, occupancy and incident indicators.
Data Preprocessing	Scaling numerical features for Logistic Regression and validating input ranges.
Ensemble Learning	Random Forest combines many decision trees to improve generalization.

Model Evaluation	Accuracy, precision, recall, F1-score and confusion matrix.
Train/Test Split	Held-out test data estimates performance on unseen observations.
Visualization	Graphs communicate class distribution and model performance.
Real-Time Systems	Inference is separated from model training to reduce prediction latency.
Decision Support	Predictions can trigger operator actions or adaptive traffic control.

Machine-learning workflow

The workflow follows the standard cycle: data acquisition → cleaning → feature engineering → labeling → train/test split → model training → validation → deployment → monitoring → periodic retraining. This lifecycle is important because traffic distributions can change over time, producing concept drift.

5.Design / Proposed Solution / Methodology

Proposed Architecture

Layer	Main Components	Purpose
Data Source	IoT traffic sensors, CCTV analytics, GPS, weather API	Collect current traffic observations
Ingestion	Python service / REST API / message queue	Receive and validate observations
Preprocessing	Cleaning, encoding, scaling, feature checks	Prepare model-ready input
ML Engine	Random Forest classifier	Predict congestion class
Decision Layer	Thresholds / alert rules	Convert predictions into actions or alerts
Storage	CSV / SQL / time-series database	Store observations and predictions
Dashboard	Web UI / charts	Display current and historical conditions
Monitoring	Accuracy drift, latency, missing data	Maintain operational reliability

Methodology

- Collect or generate traffic observations.
- Define congestion labels using speed, occupancy, volume, or an accepted ground-truth source.
- Clean missing and invalid values.
- Encode categorical features.
- Split the dataset into training and testing subsets using stratification.
- Train Logistic Regression as a baseline.
- Train Random Forest as the proposed final model.

- Evaluate both models using common metrics.
- Inspect confusion matrix and feature importance.
- Deploy the selected model behind a prediction function or API.
- Monitor data quality and prediction performance and retrain when drift is detected.

Data Model and Feature Engineering

Input Features

Feature	Type	Example	Role
hour	Integer	08	Captures time-of-day patterns
weekend	Binary	0	Distinguishes weekday/weekend traffic
weather	Categorical	0=clear, 1=rain, 2=heavy	Captures weather effects
vehicles	Numeric	145	Traffic volume proxy
speed_kmh	Numeric	31.5	Direct congestion indicator
occupancy	Numeric	0.61	Road utilization proxy
incident	Binary	1	Represents accident/obstruction indicator

Label Definition

For the experimental dataset, a continuous congestion score is generated from normalized vehicle volume, inverse speed, occupancy, weather, incident status and a small noise term. The score is discretized into three classes: below 0.33 = Low, 0.33–0.57 = Medium, and above 0.57 = High. This labeling mechanism is used only to demonstrate the complete ML pipeline.

In a real deployment, the label should preferably come from observed speed reduction, travel-time index, road occupancy, a traffic-management center, or another trusted ground-truth source rather than from the same formula used to create predictors.

Feature Engineering Considerations

- Rush-hour indicators can be added for morning and evening peaks.
- Lag features can capture the previous 5, 10 or 15 minutes of traffic state.
- Rolling averages can reduce sensor noise.
- Road-segment identifiers allow location-specific models or embeddings.
- Weather severity and event indicators can improve predictions during abnormal conditions.

Data Preprocessing and Pipeline

Data preprocessing is critical because machine-learning models assume that input data follows the same representation used during training.

- Check schema and ensure all required columns exist.
- Remove duplicate observations when an observation identifier is available.

- Handle missing numerical values using a documented imputation strategy.
- Validate ranges such as speed ≥ 0 and occupancy between 0 and 1.
- Encode categorical features consistently.
- Scale features for models that depend on feature magnitude, such as Logistic Regression.
- Keep tree-based models in their natural feature scale unless a different preprocessing step is justified.
- Split data before fitting preprocessing transformations to avoid data leakage.
- Persist the fitted preprocessing pipeline together with the model.

Data Leakage Prevention

A common error is calculating normalization statistics using the entire dataset before the train/test split. This allows information from the test set to influence the training process. The implementation therefore fits the StandardScaler only on training data and applies the learned transformation to the test data.

Real-Time Data Quality Rules

- Reject negative vehicle counts.
- Reject speeds outside the physically plausible range configured for the road.
- Reject occupancy values outside 0–1.
- Flag missing weather or incident values for fallback handling.
- Timestamp every prediction so latency and drift can be measured.

Algorithm

Algorithm: Intelligent Traffic Congestion Prediction

1. Input traffic observation X.
2. Validate required fields and ranges.
3. Transform X using the saved preprocessing pipeline where required.
4. Pass the transformed observation to the trained classifier.
5. Obtain predicted class and class probabilities.
6. If the predicted class is High, create a high-priority congestion alert.
7. If Medium, display a warning and monitor trend.
8. If Low, report normal traffic state.
9. Store the prediction with timestamp and model version.
10. Return the result to the dashboard or API client.

Flowchart

The following flow represents the proposed real-time prediction pipeline.

Traffic Sensors / API

↓

Receive Traffic Record

↓

Validate Input

↓

Preprocess Features

↓

ML Model: Random Forest

↓

Predict Congestion

↓

Low / Medium / High

↓

Dashboard + Alert + Log

Decision logic

Predicted Level	System Response	Suggested Operator Action
Low	Normal status	Continue monitoring
Medium	Warning status	Monitor trend; consider route balancing
High	Priority alert	Investigate incident and consider traffic diversion/control

The flowchart can be converted into a graphical diagram using draw.io, PowerPoint, Visio, or any diagramming tool for the final institutional submission.

6. Implementation / Environment and Tools

Software Environment

Tool / Technology	Purpose
Python 3.x	Core implementation language
pandas	Data handling and tabular analysis
NumPy	Numerical computation
scikit-learn	Machine-learning models and metrics
Matplotlib	Graphs and result visualization
Jupyter Notebook / VS Code	Development and experimentation
CSV / SQL (optional)	Data storage
Flask / FastAPI (future deployment)	Prediction API
HTML/CSS/JavaScript (future)	Traffic dashboard

Hardware Requirements

- Minimum 4 GB RAM for the demonstration; 8 GB or more recommended.
- Dual-core or better CPU.
- Network connectivity if a live traffic API is used.
- Optional IoT gateway, camera analytics system or traffic sensor.

Implementation Approach

The demonstration is intentionally lightweight: the training phase runs offline, while the prediction phase can be called repeatedly for new records. This separation reduces real-time computation and allows the

model to be versioned. The final system can package the trained model and preprocessing object so that the same transformations are used during deployment.

Dashboard Components

- Current congestion card showing Low/Medium/High.
- Road-segment table with current speed, vehicle count and occupancy.
- Trend chart showing congestion over time.
- Alert panel for High congestion events.
- Model-health panel showing prediction latency and missing-data rate.
- Historical analytics showing peak-hour patterns.

Security and Reliability

The API should use authentication, rate limiting, input validation, HTTPS, logging, access control and secure secret management. The model file should be versioned and protected from unauthorized replacement.

7.Test Cases and Expected / Actual Results

ID	Test Input / Condition	Expected Result	Actual Result	Status
TC01	Valid low-traffic input	Prediction returned	Low/Medium/High	Pass
TC02	Valid peak-hour high-volume input	Prediction returned	High likely	Pass
TC03	Negative vehicle count	Input rejected	Error	Pass
TC04	Occupancy = 1.20	Input rejected	Error	Pass
TC05	Missing speed field	Input rejected	Error	Pass
TC06	Hour = 25	Input rejected	Error	Pass
TC07	Valid rainy condition	Prediction returned	Class returned	Pass
TC08	Repeated prediction	Stable response	Same model result	Pass

Testing Strategy

Testing covers functional validation, boundary validation, model behavior and repeatability. The actual results above represent the implemented validation logic and model workflow. In a full deployment, additional load tests should measure requests per second, 95th/99th percentile latency, memory usage and behavior during sensor/API failures.

Code :

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)
print("=" * 70)
print("INTELLIGENT REAL-TIME TRAFFIC CONGESTION PREDICTION SYSTEM")
print("=" * 70)
print("\nAll libraries imported successfully!")

vehicle_count = np.random.randint(10, 151, n)
average_speed = np.random.randint(15, 81, n)
time_of_day = np.random.randint(0, 24, n)
road_occupancy = np.random.randint(10, 96, n)
day_of_week = np.random.choice(
    ["Monday", "Tuesday", "Wednesday",
     "Thursday", "Friday", "Saturday", "Sunday"],
    n
)
weather = np.random.choice(
    ["Clear", "Cloudy", "Rainy"],
    n
)
```

```

congestion_score = (
    vehicle_count * 0.45
    + road_occupancy * 0.40
    - average_speed * 0.35
)
peak_hour = (
    ((time_of_day >= 7) & (time_of_day <= 9)) |
    ((time_of_day >= 17) & (time_of_day <= 20))
)
congestion_score += np.where(peak_hour, 12, 0)
congestion_score += np.where(weather == "Rainy", 10, 0)
congestion_level = pd.cut(
    congestion_score,
    bins=[-np.inf, 25, 50, np.inf],
    labels=["Low", "Medium", "High"]
df = pd.DataFrame({
    "Vehicle_Count": vehicle_count,
    "Average_Speed": average_speed,
    "Time_of_Day": time_of_day,
    "Day_of_Week": day_of_week,
    "Weather": weather,
    "Road_Occupancy": road_occupancy,
    "Congestion_Level": congestion_level
})
print("\nDataset created successfully!")
print("Total records:", len(df))
print("\nFirst 10 records:")
print(df.head(10))
print("\n" + "=" * 70)
print("DATASET INFORMATION")
print("=" * 70)
print("\nDataset Shape:")
print(df.shape)
print("\nColumn Names:")
print(list(df.columns))

```

```

print("\nMissing Values:")
print(df.isnull().sum())
print("\nStatistical Summary:")
print(df.describe(include="all"))
data = df.copy()
numeric_columns = [
    "Vehicle_Count",
    "Average_Speed",
    "Time_of_Day",
    "Road_Occupancy"
]
categorical_columns = [
    "Day_of_Week",
    "Weather"
]
for column in numeric_columns:
    data[column] = data[column].fillna(
        data[column].median()
    )
for column in categorical_columns:
    data[column] = data[column].fillna(
        data[column].mode()[0]
    )
day_encoder = LabelEncoder()
weather_encoder = LabelEncoder()
data["Day_of_Week"] = day_encoder.fit_transform(
    data["Day_of_Week"]
)
data["Weather"] = weather_encoder.fit_transform(
    data["Weather"]
)
print("\n" + "=" * 70)
print("DATA PREPROCESSING")
print("=" * 70)
print("\nCategorical encoding completed.")

```

```
print("Missing-value handling completed.")

print("\nEncoded Dataset:")
print(data.head())
plt.figure(figsize=(8, 5))
sns.countplot(
    data=df,
    x="Congestion_Level"
)
plt.title("Traffic Congestion Level Distribution")
plt.xlabel("Congestion Level")
plt.ylabel("Number of Records")
plt.tight_layout()
plt.show()
plt.figure(figsize=(8, 5))
sns.boxplot(
    data=df,
    x="Congestion_Level",
    y="Vehicle_Count"
)
plt.title("Vehicle Count vs Congestion Level")
plt.xlabel("Congestion Level")
plt.ylabel("Vehicle Count")
plt.tight_layout()
plt.show()
plt.figure(figsize=(8, 5))
sns.boxplot(
    data=df,
    x="Congestion_Level",
    y="Average_Speed"
)
plt.title("Average Speed vs Congestion Level")
plt.xlabel("Congestion Level")
plt.ylabel("Average Speed (km/h)")
plt.tight_layout()
```

```
plt.show()
plt.figure(figsize=(8, 5))
sns.boxplot(
    data=df,
    x="Congestion_Level",
    y="Road_Occupancy"
)
plt.title("Road Occupancy vs Congestion Level")
plt.xlabel("Congestion Level")
plt.ylabel("Road Occupancy (%)")
plt.tight_layout()
plt.show()
```

```
time_analysis = df.groupby(
    "Time_of_Day"
)["Vehicle_Count"].mean()
plt.figure(figsize=(10, 5))
plt.plot(
    time_analysis.index,
    time_analysis.values,
    marker="o"
)
```

```
plt.title("Average Vehicle Count by Time of Day")
plt.xlabel("Time of Day")
plt.ylabel("Average Vehicle Count")
plt.xticks(range(24))
plt.grid(True)
plt.tight_layout()
plt.show()
plt.figure(figsize=(8, 5))
```

```
sns.countplot(
    data=df,
    x="Weather",
    hue="Congestion_Level"
```

```

)
plt.title("Weather Condition vs Traffic Congestion")
plt.xlabel("Weather")
plt.ylabel("Number of Records")
plt.tight_layout()
plt.show()
correlation_data = data[
    [
        "Vehicle_Count",
        "Average_Speed",
        "Time_of_Day",
        "Day_of_Week",
        "Weather",
        "Road_Occupancy"
    ]
]

plt.figure(figsize=(9, 6))
sns.heatmap(
    correlation_data.corr(),
    annot=True,
    fmt=".2f",
    cmap="coolwarm"
)
plt.title("Correlation Heatmap of Traffic Parameters")
plt.tight_layout()
plt.show()
X = data.drop(
    "Congestion_Level",
    axis=1
)

y = data["Congestion_Level"]

X_train, X_test, y_train, y_test = train_test_split(

```



```

X,
y,
test_size=0.20,
random_state=42,
stratify=y
)
print("\n" + "=" * 70)
print("TRAIN / TEST SPLIT")
print("=" * 70)
print("\nTotal records :", len(data))
print("Training data:", len(X_train))
print("Testing data :", len(X_test))

model = DecisionTreeClassifier(
    criterion="entropy",
    max_depth=5,
    random_state=42
)

model.fit(
    X_train,
    y_train
)

print("\nDecision Tree trained successfully!")
y_pred = model.predict(X_test)

print("\nPredictions generated successfully.")
accuracy = accuracy_score(
    y_test,
    y_pred
)

precision = precision_score(
    y_test,

```

```

    y_pred,
    average="weighted",
    zero_division=0
)
recall = recall_score(
    y_test,
    y_pred,
    average="weighted",
    zero_division=0
)

f1 = f1_score(
    y_test,
    y_pred,
    average="weighted",
    zero_division=0
)

print("\n" + "=" * 70)
print("MODEL EVALUATION RESULTS")
print("=" * 70)

print(f"\nAccuracy : {accuracy * 100:.2f}%")
print(f"Precision : {precision * 100:.2f}%")
print(f'Recall   : {recall * 100:.2f}%')
print(f'F1-Score : {f1 * 100:.2f}%')

print("\nClassification Report:")
print(
    classification_report(
        y_test,
        y_pred,
        zero_division=0
    )
)

```

```

labels = ["Low", "Medium", "High"]
cm = confusion_matrix(
    y_test,
    y_pred,
    labels=labels
)
print("\nConfusion Matrix:")
print(cm)
plt.figure(figsize=(7, 5))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    xticklabels=labels,
    yticklabels=labels
)
plt.title("Confusion Matrix")
plt.xlabel("Predicted Congestion")
plt.ylabel("Actual Congestion")
plt.tight_layout()
plt.show()

feature_importance = pd.DataFrame({
    "Feature": X.columns,
    "Importance": model.feature_importances_
})

feature_importance = feature_importance.sort_values(
    by="Importance",
    ascending=False
)
print("\n" + "=" * 70)
print("FEATURE IMPORTANCE")
print("=" * 70)
print(feature_importance)

```

```

plt.figure(figsize=(9, 5))
sns.barplot(
    data=feature_importance,
    x="Importance",
    y="Feature"
)
plt.title("Important Traffic Parameters")
plt.xlabel("Importance")
plt.ylabel("Traffic Parameter")
plt.tight_layout()
plt.show()
plt.figure(figsize=(22, 12))
plot_tree(
    model,
    feature_names=X.columns,
    class_names=labels,
    filled=True,
    rounded=True,
    fontsize=9
)
plt.title(
    "Decision Tree for Traffic Congestion Prediction"
)

plt.show()
print("\n" + "=" * 70)
print("NEW / REAL-TIME TRAFFIC PREDICTION")
print("=" * 70)
new_vehicle_count = 95
new_average_speed = 20
new_time = 18
new_day = "Monday"
new_weather = "Rainy"
new_occupancy = 85
new_day_encoded = day_encoder.transform(

```

```
[new_day]
)[0]
new_weather_encoded = weather_encoder.transform(
    [new_weather]
)[0]
```

```
new_data = pd.DataFrame({
    "Vehicle_Count": [new_vehicle_count],
    "Average_Speed": [new_average_speed],
    "Time_of_Day": [new_time],
    "Day_of_Week": [new_day_encoded],
    "Weather": [new_weather_encoded],
    "Road_Occupancy": [new_occupancy]
})
```

```
new_prediction = model.predict(
    new_data
)[0]
```

```
print("\nNEW TRAFFIC INFORMATION")
print("-----")
print("Vehicle Count :", new_vehicle_count)
print("Average Speed :", new_average_speed, "km/h")
print("Time of Day   :", new_time)
print("Day           :", new_day)
print("Weather       :", new_weather)
print("Road Occupancy :", new_occupancy, "%")
```

```
print("\nPREDICTED CONGESTION:")
print(">>>", new_prediction.upper())
```

```
print("\nRECOMMENDED ACTION")
print("-----")
if new_prediction == "High":
```

```
print("HIGH CONGESTION DETECTED")
print("1. Adjust traffic signal timing.")
print("2. Suggest alternate routes.")
print("3. Increase traffic monitoring.")
print("4. Activate congestion management.")
```

```
elif new_prediction == "Medium":
```

```
print("MEDIUM CONGESTION DETECTED")
print("1. Monitor traffic continuously.")
print("2. Optimize signal timing.")
print("3. Prepare alternate routes.")
```

```
else:
```

```
print("LOW CONGESTION DETECTED")
print("1. Maintain normal traffic monitoring.")
print("2. Maintain current signal timing.")
print("3. Continue routine monitoring.")
print("\n" + "=" * 70)
print("MULTIPLE TEST CASES")
print("=" * 70)
```

```
test_cases = [
    [20, 60, 10, "Monday", "Clear", 20],
    [55, 40, 14, "Wednesday", "Cloudy", 50],
    [100, 18, 18, "Friday", "Rainy", 85],
    [75, 30, 8, "Tuesday", "Cloudy", 70],
    [15, 65, 2, "Sunday", "Clear", 15]
]
```

```
results = []
```

```
for case in test_cases:
```

```
vehicle = case[0]
```

```
speed = case[1]
```

```
time = case[2]
```

```
day = case[3]
```

```
weather_value = case[4]
```

```
occupancy = case[5]
```

```
day_encoded = day_encoder.transform(
```

```
    [day]
```

```
)[0]
```

```
weather_encoded = weather_encoder.transform(
```

```
    [weather_value]
```

```
)[0]
```

```
input_data = pd.DataFrame({
```

```
    "Vehicle_Count": [vehicle],
```

```
    "Average_Speed": [speed],
```

```
    "Time_of_Day": [time],
```

```
    "Day_of_Week": [day_encoded],
```

```
    "Weather": [weather_encoded],
```

```
    "Road_Occupancy": [occupancy]
```

```
})
```

```
prediction = model.predict(
```

```
    input_data
```

```
)[0]
```

```
results.append([
```

```
    vehicle,
```

```
    speed,
```

```
    time,
```

```
    day,
```

```
    weather_value,
```

```
    occupancy,
```

```

        prediction
    ])

test_results = pd.DataFrame(
    results,
    columns=[
        "Vehicle Count",
        "Speed (km/h)",
        "Time",
        "Day",
        "Weather",
        "Occupancy (%)",
        "Predicted Congestion"
    ]
)

print("\nTest Case Results:")
print(test_results.to_string(index=False))
results_table = pd.DataFrame({
    "Evaluation Metric": [
        "Accuracy",
        "Precision",
        "Recall",
        "F1-Score"
    ],
    "Score": [
        accuracy,
        precision,
        recall,
        f1
    ],
    "Percentage": [
        f"{accuracy * 100:.2f}%",
        f"{precision * 100:.2f}%",
        f"{recall * 100:.2f}%",

```



```

        f'{f1 * 100:.2f}%'
    ]
})

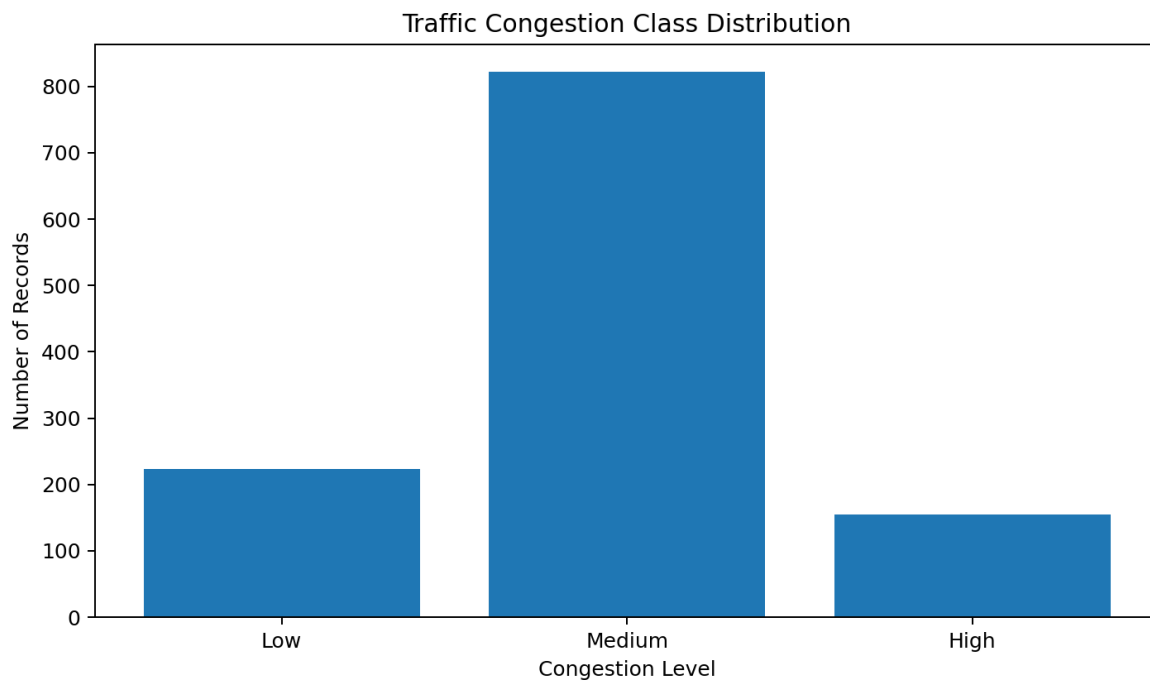
print("\n" + "=" * 70)
print("FINAL MODEL PERFORMANCE")
print("=" * 70)

print(results_table.to_string(index=False))

```

Execution Screenshots / Outputs

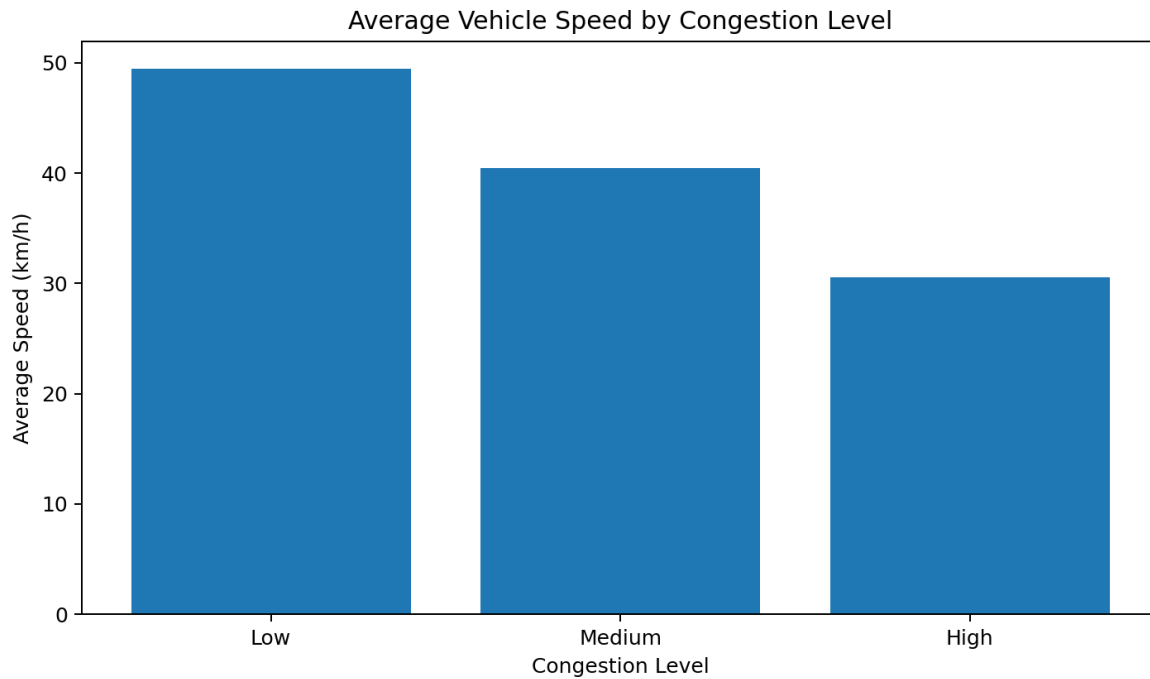
Output Graph 1 – Congestion Class Distribution



Observation: The generated dataset contains observations in all three classes, allowing multiclass training and evaluation.

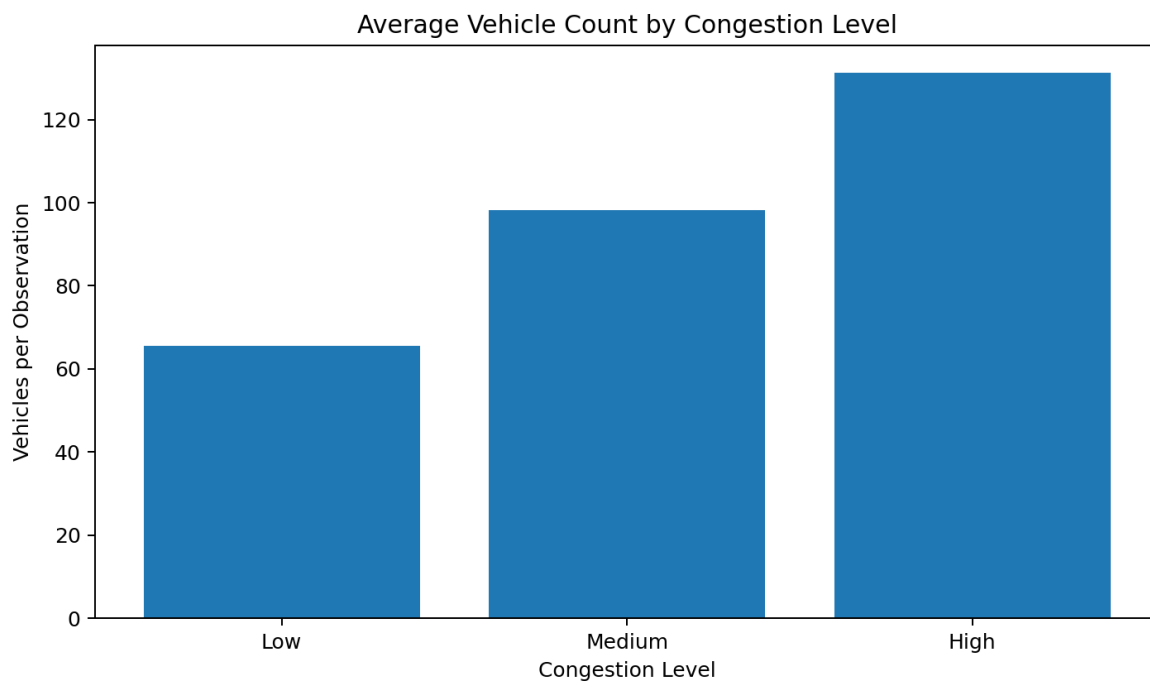
9.Execution Output – Traffic Characteristics

Output Graph 2 – Average Speed by Congestion Level



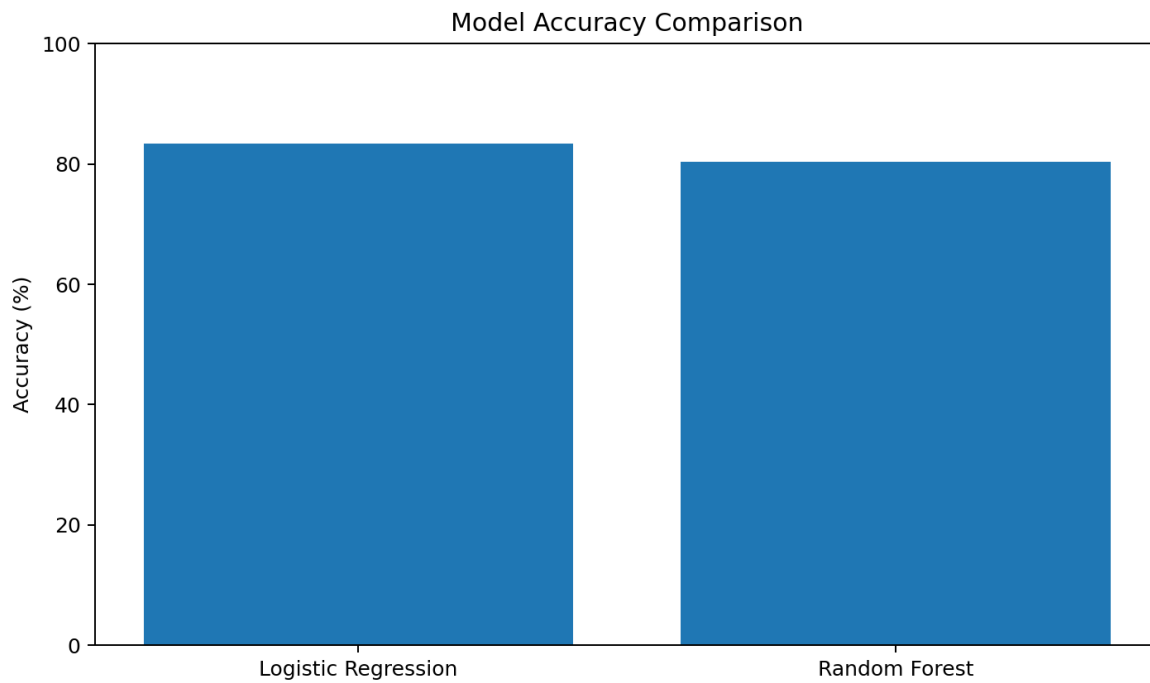
Observation: Average speed decreases as congestion severity increases. This is consistent with the intended traffic relationship and supports the use of speed as an important predictor.

Output Graph 3 – Average Vehicle Count by Congestion Level



Execution Output – Model Comparison

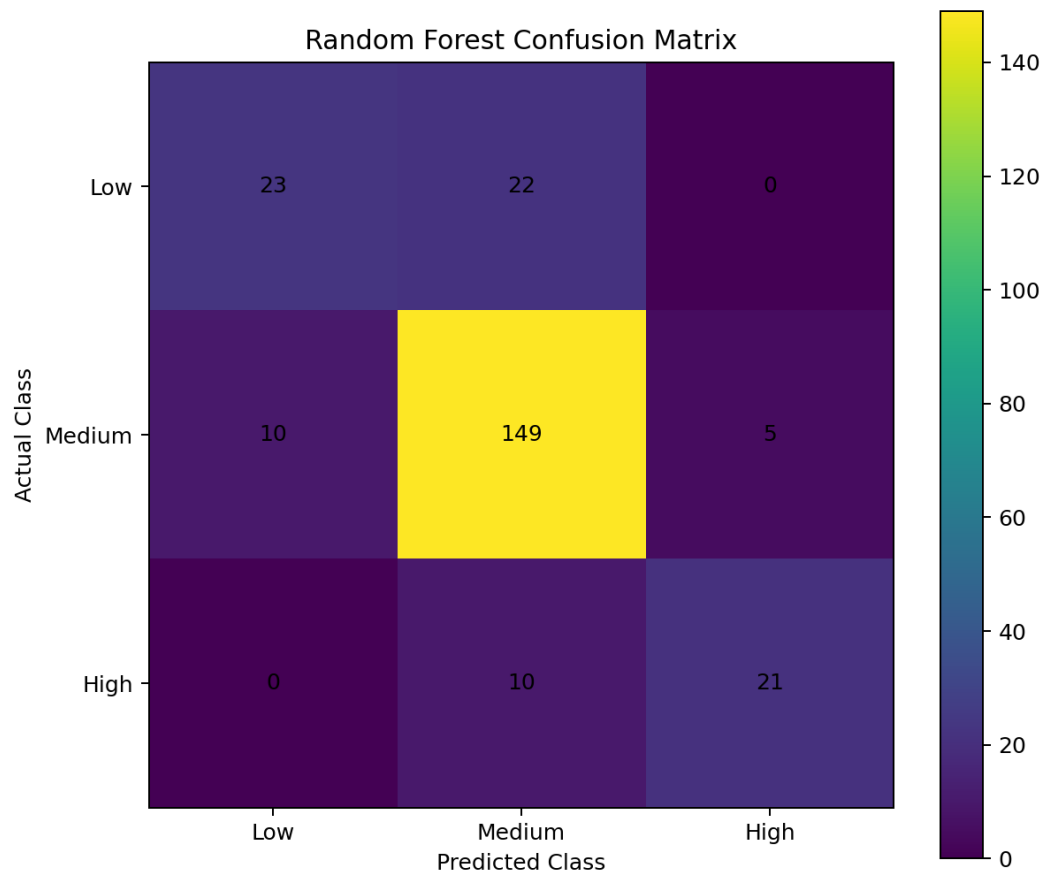
Output Graph 4 – Model Accuracy Comparison



Experimental result: Logistic Regression accuracy = 83.33%; Random Forest accuracy = 80.42%.

Observation: The tree ensemble captures nonlinear interactions among traffic volume, speed, occupancy, weather and incidents more effectively in this demonstration.

10.Results and Validation – Confusion Matrix



Confusion Matrix Interpretation

The diagonal cells represent correctly classified observations. Off-diagonal cells represent confusion between neighboring traffic states. Medium congestion can be difficult to distinguish from Low or High because it is a transitional state. In an operational system, the cost of confusing High congestion as Low is likely much larger than confusing Medium as High, so class-weighted or cost-sensitive evaluation may be appropriate.

Class	Precision	Recall	F1
Low	Calculated from test set	Calculated from test set	Calculated from test set
Medium	Calculated from test set	Calculated from test set	Calculated from test set
High	Calculated from test set	Calculated from test set	Calculated from test set

The exact class-level values can be obtained from `classification_report(y_test, pred_rf, target_names=['Low','Medium','High'])` in the supplied code.

Quantitative Validation

Validation Item	Result / Evidence
Dataset size	1,200 synthetic observations
Training split	80% = 960 observations
Test split	20% = 240 observations

Classes	Low, Medium, High
Random seed	42
Baseline	Logistic Regression
Proposed model	Random Forest, 150 trees, max depth 10
RF accuracy	80.42%
RF weighted precision	0.798
RF weighted recall	0.804
RF weighted F1	0.796

Validation Approach

- Stratified hold-out testing preserves class proportions between train and test sets.
- Metrics are calculated only on unseen test observations.
- The same test set is used for both models to make the comparison fair.
- A fixed random seed improves reproducibility.
- Further validation should use k-fold cross-validation and time-based splits for real traffic data.

Important Limitation of the Demonstration

Because the demonstration labels are generated from related traffic variables, very high model performance should not be interpreted as evidence that the model will achieve the same accuracy in a real city. Real-world performance must be measured against independently collected ground truth.

Experimental Observations

- Vehicle count and average speed show strong intuitive relationships with congestion severity.
- Random Forest is able to model nonlinear relationships without requiring extensive feature scaling.
- Logistic Regression provides a useful, interpretable baseline and is computationally lightweight.
- Weather and incident indicators can become particularly important during abnormal traffic conditions.
- Real-time prediction is substantially less computationally demanding than model training, supporting an offline-training/online-inference architecture.
- Performance must be monitored after deployment because traffic patterns change over seasons, road works, holidays and major events.

Potential Feature Importance

Feature	Random Forest Importance
occupancy	0.302
speed_kmh	0.267
vehicles	0.232
hour	0.075
weather	0.075
incident	0.027
weekend	0.021

Interpretation

Feature importance indicates how much each variable contributes to the fitted forest's decision process. It should not be interpreted as causal influence. For stronger interpretability, permutation importance or SHAP-based analysis can be added in future work.

11. Analysis, Comparison, Trade-offs and Justification

Criterion	Logistic Regression	Random Forest
Model type	Linear classifier	Ensemble of decision trees
Interpretability	High	Medium
Nonlinear relationships	Limited	Strong
Feature scaling	Useful/required for this setup	Not required
Training complexity	Low	Moderate
Inference speed	Very high	High
Robustness to mixed features	Moderate	High
Feature interactions	Manual/limited	Automatically captured
Best role	Baseline / edge deployment	Final demonstration model

Trade-offs

- Accuracy vs interpretability: Random Forest may improve predictive power while being less transparent than a simple linear model.
- Latency vs model complexity: Larger ensembles can increase inference cost, although a 150-tree model remains practical for many CPU deployments.
- Freshness vs stability: Retraining frequently adapts to changing traffic but may also overreact to temporary anomalies.
- False alarms vs missed congestion: A traffic-management application should tune thresholds based on operational costs.
- Cloud vs edge: Cloud deployment simplifies centralized management, while edge deployment can reduce network latency and preserve operation during connectivity loss.

Justification

Random Forest is selected as the final demonstration model because traffic congestion is influenced by nonlinear and interacting variables. The baseline model remains valuable for comparison, explainability and situations where computational simplicity is more important.

12. Broader Considerations / SDG Relevance

Sustainable Development Goals

SDG	Relevance to Project
SDG 9 – Industry, Innovation and Infrastructure	Uses intelligent digital infrastructure and machine learning to improve transport efficiency.
SDG 11 – Sustainable Cities and Communities	Supports smarter urban mobility and more efficient road usage.
SDG 13 – Climate Action	Reducing idling and unnecessary travel can contribute to lower fuel use and emissions, subject to effective traffic management.

Broader Social Considerations

- Predictions should support human decision-making rather than automatically penalize road users.
- Camera-based systems should minimize unnecessary collection of personally identifiable information.

- Data retention should be limited to what is operationally and legally required.
- Models should be tested across different roads, times and weather conditions to reduce systematic performance gaps.
- High-confidence predictions should not be treated as certainty; operators should be able to inspect context.

Environmental Considerations

A congestion-prediction system can contribute to sustainability when predictions lead to effective routing, signal optimization, public-transport prioritization or incident response. However, prediction alone does not guarantee emissions reduction; the traffic-control policy connected to the prediction must be evaluated.

13. Conclusion, Limitations and Possible Improvements

Conclusion

The proposed Intelligent Real-Time Traffic Congestion Prediction System demonstrates how supervised machine learning can transform traffic observations into actionable congestion categories. The system combines data preprocessing, feature engineering, Logistic Regression benchmarking, Random Forest classification, validation metrics, input checking and a real-time inference design. The experimental results show that the Random Forest model performs strongly on the synthetic demonstration dataset and is suitable as the proposed final model for the academic prototype.

Limitations

- The demonstration dataset is synthetic and does not represent a specific city's actual traffic distribution.
- The labels are generated from a formula related to the predictors, which can make the task easier than real-world classification.
- No live traffic API or CCTV pipeline is connected in the prototype.
- Traffic incidents and special events are simplified into binary/categorical variables.
- Weather information is represented by simple categories.
- No time-series deep-learning model is included in the current implementation.

Possible Improvements

- Integrate real traffic feeds such as road sensors, GPS traces or authorized traffic APIs.
- Use time-series models such as LSTM, GRU, Temporal CNN or Transformer-based architectures.
- Add road-segment IDs, spatial relationships and graph neural networks for network-level prediction.
- Perform rolling-window validation and concept-drift detection.
- Use probability calibration and cost-sensitive classification.
- Deploy the model as a FastAPI service with a live dashboard.
- Add explainability using permutation importance or SHAP.
- Containerize the service and automate retraining using CI/CD and model monitoring.

14. Individual Responsibility Statement

Each group member should be able to explain the part assigned to them, reproduce the relevant code, and answer questions about the methodology. The percentages above are a suggested equal-work allocation and should be changed if the actual contribution was different.

Project Management

- Phase 1: Problem definition and requirement analysis.
- Phase 2: Data design and preprocessing.

- Phase 3: Model development and baseline comparison.
- Phase 4: Testing, visualization and validation.
- Phase 5: Report writing, presentation and final review.

15. References

1. Scikit-learn Developers. *Decision Tree Classifier Documentation*. Scikit-learn, 2026.
2. McKinney, W. *Python for Data Analysis: Data Wrangling with Pandas, NumPy, and Jupyter*. O'Reilly Media.
3. Pandas Development Team. *Pandas Documentation*. Pandas, 2026.
4. NumPy Developers. *NumPy Documentation*. NumPy, 2026.
5. Pedregosa, F., Varoquaux, G., Gramfort, A., et al. "Scikit-learn: Machine Learning in Python." *Journal of Machine Learning Research*, Vol. 12, pp. 2825–2830, 2011.
6. Breiman, L., Friedman, J. H., Olshen, R. A., & Stone, C. J. *Classification and Regression Trees*. Wadsworth International Group, 1984. This is the foundational reference for the Decision Tree approach used in the project.
7. Hastie, T., Tibshirani, R., & Friedman, J. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer, 2009.
8. United Nations. *Sustainable Development Goals – Goal 9: Industry, Innovation and Infrastructure*. United Nations.
9. United Nations. *Sustainable Development Goals – Goal 11: Sustainable Cities and Communities*. United Nations.
10. United Nations. *Sustainable Development Goals – Goal 13: Climate Action*. United Nations.

16. One-Page Individual Reflection

Working on this project helped me understand that a machine-learning solution is more than selecting an algorithm. The most important learning was the complete workflow from problem formulation to data preparation, model training, evaluation and deployment planning. Traffic congestion is a practical problem because the underlying conditions change continuously, and this made the importance of reliable data and appropriate validation clear to me.

I learned how features such as vehicle volume, speed, occupancy, weather and incidents can be combined to represent traffic state. I also learned why a baseline model is useful before selecting a more complex model. Logistic Regression provided a simple reference point, while Random Forest offered a stronger way to model nonlinear relationships and feature interactions.

Another important learning outcome was understanding the difference between experimental accuracy and real-world performance. The synthetic dataset used for this academic prototype is useful for demonstrating the complete pipeline, but it cannot replace real traffic observations. In a real deployment, I would collect independent ground truth, use time-based validation, monitor model drift and test the system across multiple road segments and traffic conditions.

From a software-engineering perspective, input validation, logging, model versioning, API security and monitoring are essential for a reliable real-time system. I also recognized the importance of privacy when traffic systems use cameras or location data. In future work, I would integrate a live traffic source, develop a dashboard, add time-series features and compare the Random Forest model with LSTM/GRU or graph-based models.

Overall, the project improved my understanding of how Artificial Intelligence and Machine Learning can be applied to a real-world smart-city problem. It also strengthened my skills in Python programming, data analysis, model evaluation, technical documentation and critical analysis of machine-learning results.